



Week-2

1D and 2D Dynamic Safe Array, Jagged Array

DATE: 25-29/9/2025

INSTRUCTOR: FIZZA AQEEL

Vectors

- ▶ A **vector** in C++ is a **dynamic array** provided by the **Standard Template Library (STL)**.
- ▶ Unlike normal arrays, vectors can **resize themselves automatically** when elements are inserted or deleted.
- ▶ **Key Features**
 - ▶ Stored in **contiguous memory** (like arrays).
 - ▶ **Dynamic resizing** (no need to declare size in advance).
 - ▶ Provides many **built-in functions** for insertion, deletion, traversal.
 - ▶ Random access in **$O(1)$** time (like arrays).
 - ▶ Insertion/deletion at end: **$O(1)$** (amortized).
 - ▶ Insertion/deletion in middle: **$O(n)$** (because elements shift).

Vectors- code example

```
//declaration
#include <vector>
using namespace std;

vector<int> v1;           // Empty vector of integers
vector<int> v2(5);        // Vector of size 5, default
                           initialized
vector<int> v3(5, 10);    // Vector of size 5, all elements =
                           10
vector<int> v4 = {1, 2, 3}; // Initialization list
```

Key Points:

- Use Array if data size is fixed & you need speed.
- Use Vector if you need dynamic resizing + random access.
- Use List if you need frequent insertions/deletions in the middle.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    // Insert elements
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    // Access elements
    cout << "First element: " << v[0] << endl;
    cout << "Last element: " << v.back() << endl;
    // Size and capacity
    cout << "Size: " << v.size() << endl;
    cout << "Capacity: " << v.capacity() << endl;
    // Iterating
    cout << "Elements: ";
    for(int x : v)
        cout << x << " ";
    // Remove last element
    v.pop_back();

    cout << "\nAfter pop_back(), Size: " << v.size() << endl;
}
```

Array vs List vs Vector in C++

Feature	Array (C-style / <code>std::array</code>)	Vector (<code>std::vector</code>)	List (<code>std::list</code>)
Memory Storage	Contiguous (fixed size)	Contiguous (resizable)	Non-contiguous (doubly linked nodes)
Size	Fixed at compile-time	Dynamic (grows/shrinks automatically)	Dynamic (grows/shrinks automatically)
Access Time	$O(1)$ (direct index)	$O(1)$ (direct index)	$O(n)$ (must traverse)
Insertion/Deletion (end)	Not possible (fixed size)	$O(1)$ amortized (<code>push_back</code> , <code>pop_back</code>)	$O(1)$ if iterator known
Insertion/Deletion (middle)	Expensive (manual shifting)	$O(n)$ (shifts elements)	$O(1)$ if iterator known (just re-link)
Iteration Speed	Fastest (cache-friendly)	Fast (cache-friendly)	Slower (pointer chasing)
Memory Usage	Very low	Slight overhead (capacity mgmt)	High (extra pointers per node)
Best Use Case	Fixed-size data (like 7 days, 12 months)	Dynamic arrays, frequent random access	Frequent insertions/deletions in middle

Rule Of Three

- ▶ The Rule of Three in C++ states that if a class explicitly defines any of the following, it should explicitly define all three:
 - ▶ Destructor: Responsible for releasing resources acquired by the object, especially dynamically allocated memory.
 - ▶ Copy Constructor: Defines how a new object is created as a copy of an existing object.
 - ▶ Copy Assignment Operator: Defines how an existing object is assigned the value of another existing object.
- ▶ This rule is crucial for classes that manage dynamic resources to prevent issues like shallow copies, memory leaks, and double deletions.

```
~ClassName() {  
    // free dynamically allocated memory  
}
```

```
ClassName(const ClassName &other) {  
    // deep copy code here  
}
```

```
ClassName& operator=(const ClassName &other) {  
    if(this != &other) {    // check self-assignment  
        // free old memory  
        // allocate new memory  
        // copy values  
    }  
    return *this;  
}
```

Why Rule Of Three?

- ▶ Because if your class manages resources like **dynamic memory, file handles, or network connections**, then the default implementations provided by the compiler may cause:
- ▶ **Shallow copies** (multiple objects pointing to same memory → double deletion error).
- ▶ **Memory leaks** (if destructor isn't properly handling deallocation).
- ▶ **Unexpected behavior** (assignment overwriting without cleanup).

Rule Of Three

```
#include <iostream>
using namespace std;

class MyClass {
    int* data;
public:
    // Constructor
    MyClass(int value) {
        data = new int(value);
    }

    // Destructor
    ~MyClass() {
        delete data;
    }

    // Copy Constructor
    MyClass(const MyClass& other) {
        data = new int(*other.data); // deep copy
```

```
// Copy Assignment Operator
MyClass& operator=(const MyClass& other) {
    if(this != &other) {
        delete data;
        data = new int(*other.data);
    }
    return *this;
}

void display() { cout << "Value: " << *data << endl; }
};

int main() {
    MyClass obj1(10);
    MyClass obj2 = obj1; // Copy constructor
    obj2.display();

    MyClass obj3(20);
    obj3 = obj1; // Copy assignment
    obj3.display();
}
```

```
//output
Value: 10
Value: 10
```

1D and 2D Dynamic Safe Array

- ▶ Think of memory array used as a **hotel**:
- ▶ Each **room** is a memory slot.
- ▶ An **array** is like booking rooms in a straight line (contiguous).
- ▶ A **safe array** is like booking rooms **with a guard at the door**, ensuring you don't enter the wrong room.
- ▶ A **dynamic safe array** is like telling the hotel **at runtime** how many rooms you want (not fixed at design time).

1D Dynamic Safe Array

Imagine a locker row in a gym:

- ▶ You decide how many lockers you need while entering (runtime).
- ▶ Each locker is protected, so you can't wrongly access locker -1 or locker 999 when only 10 exist.

1D and 2D Dynamic Safe Array

2D Dynamic Safe Array

- ▶ Think of a chessboard: Rows = files, Columns = ranks.
- ▶ You can create any size board dynamically.
- ▶ A guard ensures you don't move a piece outside the board.

1D and 2D Dynamic Safe Array

```
#include <iostream>
using namespace std;

class Safe1D {
    int *arr; // pointer to hold array
    int size; // size of array
public:
    Safe1D(int n) { size=n; arr=new int[size]; // allocate memory}
    void set(int i,int val) {
        if(i>=0 && i<size) arr[i]=val;
        else cout<<"Index Error\n";
    }
    int get(int i) {
        if(i>=0 && i<size) return arr[i];
        cout<<"Index Error\n"; return -1;
    }
    ~Safe1D(){ delete[] arr; }
};

int main() {
    Safe1D a(3);
    a.set(0,10); a.set(1,20);
    cout<<a.get(1)<<endl;           // 20
    a.set(5,50);                    // Index Error
}
```

```
class Safe2D {
    int **arr, rows, cols;
public:
    Safe2D(int r,int c) {
        rows=r; cols=c;
        arr=new int*[rows];
        for(int i=0;i<rows;i++) arr[i]=new int[cols];
    }
    void set(int i,int j,int val) {
        if(i>=0&&i<rows&&j>=0&&j<cols) arr[i][j]=val;
        else cout<<"Index Error\n";
    }
    int get(int i,int j) {
        if(i>=0&&i<rows&&j>=0&&j<cols) return arr[i][j];
        cout<<"Index Error\n"; return -1;
    }
    ~Safe2D(){ for(int i=0;i<row
s;i++) delete[] arr[i]; delete[] arr; }
};

int main() {
    Safe2D m(2,2);
    m.set(0,0,11); m.set(1,1,22);
    cout<<m.get(1,1)<<endl;           // 22
    m.set(3,3,99);                   // Index Error
}
```

Jagged Array

- ▶ Imagine a **parking lot**:
- ▶ A **2D array** is like a parking lot where **each row has the same number of slots** (e.g., every row has exactly 5 cars).
- ▶ A **jagged array** is a parking lot where **each row can have a different number of slots** — row 1 may have 3 cars, row 2 may have 10, row 3 just 1.
- ▶ In short:
- ▶ **2D array** = rectangular grid (fixed columns).
- ▶ **Jagged array** = uneven rows (like stairs, not a rectangle).

Jagged Array in C++

- ▶ C++ doesn't directly support jagged arrays like C# or Java, but we can **simulate them** using pointers or vectors.

Method 1: Using Pointers

```
#include <iostream>
using namespace std;

int main() {
    int* jagged[3]; // 3 rows

    // each row has different column size
    jagged[0] = new int[2]; // row 0 → 2 cols
    jagged[1] = new int[4]; // row 1 → 4 cols
    jagged[2] = new int[3]; // row 2 → 3 cols

    // initialize values
    int val = 1;
    for(int i=0; i<3; i++) {
        for(int j=0; j<(i+2); j++) { // row size = i+2
            jagged[i][j] = val++;
        }
    }

    // print
    for(int i=0; i<3; i++) {
        for(int j=0; j<(i+2); j++) {
            cout << jagged[i][j] << " ";
        }
        cout << endl;
    }
}
```

```
//output
1 2
3 4 5 6
7 8 9
```

Method 2: Using Vectors

This method consider easier and safer for implementing jagged arrays in c++.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<vector<int>> jagged = {
        {1, 2},
        {3, 4, 5, 6},
        {7, 8, 9}
    };

    for(auto &row : jagged) {
        for(int val : row) cout << val << " ";
        cout << endl;
    }
}
```

```
//output
1 2
3 4 5 6
7 8 9
```

Example 2: Using Vectors

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int students;
    cout << "Enter number of students: ";
    cin >> students;

    vector<vector<int>> marks(students); // jagged array

    // Input marks for each student
    for (int i = 0; i < students; i++) {
        int subjects;
        cout << "\nEnter number of subjects for Student " << i+1 << ": ";
        cin >> subjects;

        marks[i].resize(subjects); // allocate space for this student's subjects

        cout << "Enter marks: ";
        for (int j = 0; j < subjects; j++) {
            cin >> marks[i][j];
        }
    }
}
```

```
// Display all marks
cout << "\n--- Student Marks ---\n";
for (int i = 0; i < students; i++) {
    cout << "Student " << i+1 << ": ";
    for (int j = 0; j < marks[i].size(); j++) {
        cout << marks[i][j] << " ";
    }
    cout << endl;
}
}
```

```
//Output
Enter number of students: 3

Enter number of subjects for Student 1: 2
Enter marks: 85 90

Enter number of subjects for Student 2: 4
Enter marks: 78 88 92 95

Enter number of subjects for Student 3: 3
Enter marks: 80 70 75

--- Student Marks ---
Student 1: 85 90
Student 2: 78 88 92 95
Student 3: 80 70 75
```